Professores:

Celso Giusti

Daniel Manoel Filho

Marlon Palata Fanger Rodrigues

MICROPYTHON

SENAI

MICROPYTHON

O que é MicroPython?

E a versão leve do Python 3 que roda em placas como:

- ESP32
- ESP8266
- Raspberry Pi Pico.

Uso: Ideal para projetos de automação, sensores, motores, LEDs, IoT e sistemas embarcados.

Vantagem: Código enxuto, fácil de aprender, funciona **direto no microcontrolador.**

DOCUMENTAÇÃO:
https://docs.micropython.org/en/v1.15/library/

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython

No **MicroPython**, **bibliotecas** são conjuntos de funções e classes prontas que facilitam o uso de recursos específicos do microcontrolador, como manipulação de tempo, matemática, controle de hardware (pinos, sensores), entre outros.

Vamos iniciar nessa aula com dois dos mais importantes Módulos/bibliotecas:

- **Machine**
- **utime (ou time)**

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Machine

O **Módulo/biblioteca Machine** é uma das mais importantes no **MicroPython**. Ela permite acesso direto ao hardware da placa, como **pinos**:

- **GPIO**
- **ADC** (conversão analógica)
- **PWM** (modulação por largura de pulso)
- **Comunicação**:
  - I2C
  - SPI
  - UART

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Machine

1. GPIO (Pin, IN e OUT)

- **O que é?:** São pinos digitais configuráveis como entrada (IN) ou saída (OUT).
- **Por que usar?:** Permite interagir com componentes eletrônicos, como botões, LEDs e sensores.

Exemplo:

```
#Importa o comando Pin originando-se do módulo Machine
from machine import Pin
led = Pin(15, Pin.OUT) #Define o LED e sua respectiva porta
led.value(1)  # Liga o LED
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Machine

2. ADC (Conversor Analógico para Digital)

- **O que é?:** Permite ler sinais analógicos (Ex: 0V a 3V) e converter esses valores em dados digitais (0 a 65535)

- **Por que usar?:** Serve para medir grandezas variáveis, como luz, temperatura ou posição, e transformá-las em números compreensíveis pelo código.

Exemplo:

```
#Importa o comando ADC originando-se do módulo Machine
from machine import ADC
sensor = ADC(26) # Declara o sensor e sua respectiva porta
valor = sensor.read_u16() # Armazena o valor lido no sensor
print(valor)  # Exibe valor entre 0 e 65535
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Machine

3. PWM (Sinal de modulação por largura de pulso)

- **O que é?:** Simula uma saída analógica usando um sinal digital que oscila rapidamente.
- **Por que usar?:** Ideal para controlar a intensidade de LEDs, velocidade de motores ou posição de servos.

Exemplo:

```
from machine import Pin, PWM
led = PWM(Pin(15)) # Define o LED e sua respectiva porta, porém o pino sendo PWM
led.freq(1000) # Define a frequência que será manipulada
led.duty_u16(32768)  # 50% do brilho
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Machine

4. I2C – Comunicação Serial

- **O que é?:** Um protocolo de comunicação que usa apenas dois fios para conectar vários dispositivos.

- **Por que usar?:** É ideal quando você precisa conectar vários dispositivos (como sensores e displays) e não quer usar muitos pinos.

  - **Exemplo de uso:** Conectar sensores de temperatura, displays LCD, ou EEPROMs.

```
from machine import I2C, Pin  # Importa a classe I2C
# Define o pino 17 como SCL (clock) e o pino 16 como SDA (dados), frequencia definida como 400kHz
i2c = I2C(0, scl=Pin(17), sda=Pin(16), freq=400000)
# Realiza uma varredura e retorna uma lista com os endereços I2C encontrados
dispositivos = i2c.scan()
print(dispositivos)  # Exibe os endereços dos dispositivos encontrados
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Machine

5. SPI – Comunicação SPI

- **O que é?:** Um protocolo de comunicação que usa mais fios, mas é mais rápido que o I2C.
- **Por que usar?:** Se você precisa de maior velocidade para comunicação, como em cartões SD ou displays mais complexos.
    - **Exemplo de uso:** Cartões de memória, sensores rápidos e displays gráficos.

```
from machine import SPI, Pin  # Importa a classe SPI
# baudrate = 1.000.000 bps (1 MHz), polarity = 0: sinal de clock,
#phase = 0: dados são capturados na borda de subida do clock,
#sck = pino 18 (clock), mosi = pino 19 (envio), miso = pino 16 (recebimento)
spi = SPI(0, baudrate=1000000, polarity=0, phase=0, sck=Pin(18), mosi=Pin(19), miso=Pin(16))
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Machine

6. UART – Comunicação Serial Padrão

- **O que é?:** Comunicação serial entre dois dispositivos, como um microcontrolador e um módulo Bluetooth.

- **Por que usar?:** Simples e direto, ótimo para enviar e receber dados entre dois dispositivos, sem complicação.

  - **Exemplo de uso:** Comunicação com módulos como Bluetooth, GPS ou conexão serial com o computador.

```
from machine import UART  # Importa a classe UART da biblioteca machine
# Cria um objeto UART no canal 0 (existem dois UARTs: 0 e 1)
uart = UART(0, baudrate=9600)
# Envia uma mensagem pela UART
uart.write('Olá, mundo!')  # Transmite a string via porta serial (TX)
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Utime

A **biblioteca utime** permite o uso de funções relacionadas ao tempo, como:

1. **Atrasos (sleep):**

   - **O que é?:** Usamos para pausar o programa por um tempo determinado.

   - **Por que usar?:** Às vezes, você precisa que o microcontrolador faça algo, e depois de um tempo ele faz outra coisa. O atraso ajuda a dar esse "tempo de espera".

Exemplo:

```
from utime import sleep # Utiliza a biblioteca utime para importar comandos de tempo

print("Aguardando 2 segundos...")  # Mensagem antes da pausa
sleep(2)  # Pausa o programa por 2 segundos
print("Pronto!")  # Continua após a pausa
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Utime

1.1. Atrasos (sleep - ms e us):

```
from utime import sleep, sleep_ms, sleep_us
# Utiliza a biblioteca utime para importar comandos de tempo

sleep_ms(500)  # Pausa por 500 milissegundos (meio segundo)
sleep_us(200)  # Pausa por 200 microssegundos (0.0002 segundos)
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Utime

2. Marcação de tempo (marcar o tempo) - ticks_ms() + ticks_diff()

- **O que é?:** Serve para medir o tempo que passou entre duas ações.
- **Por que usar?:** Quando você quer saber quanto tempo levou para fazer algo, como medir o tempo de execução de uma tarefa.

Exemplo:

```
import utime # Importa a biblioteca de tempo
inicio = utime.ticks_ms()  # Salva o tempo atual em milissegundos
utime.sleep(1)             # Aguarda 1 segundo
fim = utime.ticks_ms()     # Salva o tempo após a pausa
print("Tempo decorrido:", utime.ticks_diff(fim, inicio), "ms")
# ticks_diff calcula a diferença entre dois ticks
```

MICROPYTHON

Módulos/Bibliotecas Padrão Micropython: Utime

3. Data e hora local - localtime() e mktime()

- **O que é?:** Serve para trazer: Data (completo - dia, mês e ano) e hora de acordo com a sua localidade. (não é **preciso** no Wokwi)

- **Por que usar?:** Para que, de acordo com essas informações poder configurar equipamentos para executar tarefas em dias específicos. (captar informações, ligar, desligar, etc.)

Exemplo:

```
import utime
tempo = utime.localtime()  # Retorna a data e hora atual como tupla (ano, mês, dia, etc.)
print(tempo)  # Ex: (2025, 5, 3, 10, 30, 0, 5, 123)

segundos = utime.mktime(tempo)  # Converte essa data para o número de segundos desde 1970
print(segundos)
```

MICROPYTHON

Diferença de usar: import e from, import

Forma	Vantagens	Desvantagens
import ...	**- Mais explícito (bom para iniciantes)-** Permite ver de onde vem cada comando	**- Código mais verboso-** Toda chamada precisa de prefixo
from ... **import** ...	**- Código mais limpo e curto-** Ideal quando quer diminuir o uso de recursos	**-** Pode causar confusão se muitos nomes forem importados diretamente

MICROPYTHON

Diferença de usar: import e from, import

Qual é melhor?

- **Para aprendizado** ou aulas iniciais:

  import ... é melhor, pois ajuda o aluno a entender de onde vêm os comandos.

- **Para código mais limpo**, curto e prático:

  from ... **import** ... é melhor, especialmente em projetos maiores com muitos recursos.

MICROPYTHON

Resumo

✅ **machine =** Biblioteca principal para acessar os pinos e recursos da placa (GPIO, ADC, PWM).

✅ **utime =** Biblioteca para controlar tempo (pausa, tempo decorrido, etc).

MICROPYTHON

Referências

MICROPYTHON.ORG. **machine – functions related to the hardware.** MicroPython Documentation, 2024. Disponível em: https://docs.micropython.org/en/latest/library/machine.html. Acesso em: 3 maio 2025.

MICROPYTHON.ORG. **utime – time-related functions.** MicroPython Documentation, 2024. Disponível em: https://docs.micropython.org/en/latest/library/utime.html. Acesso em: 3 maio 2025.

BARENDSE, Ben. **Exploring Raspberry Pi Pico with MicroPython: A Hands-on Guide**. 1. ed. Independently published, 2022.

FOTO DA ESCOLA

ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebok

/senaisp.itu

Site

https://sp.senai.br/unidade/itu/